

○ AuthController.java

```
package myproject; import
org.springframework.stereotype.Controller; import
org.springframework.web.bind.annotation.*;
@Controller public class
AuthController {
    @GetMapping("/login")
    public String loginPage() {
        return "login";
    }
    @PostMapping("/login")    public String login(@RequestParam
String username,                @RequestParam String password)
{    if(username.equals("admin") && password.equals("admin"))
{
        return "redirect:/";
    }    return
"login";
}
}
```

○ InventorySystemmApplication.java

```
package myproject; import
org.springframework.boot.SpringApplication; import
org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication public class
InventorySystemmApplication {    public
static void main(String[] args) {
        SpringApplication.run(InventorySystemmApplication.class, args);
    }
}
```

○ Product.java

```
package myproject; import
jakarta.persistence.*;
@Entity
public class Product {
    @Id
    @GeneratedValue(strategy =
GenerationType.IDENTITY) private int id; private
String name; private double price; private int quantity;
// Constructors public Product() {} public
Product(String name, double price, int quantity) {
this.name = name; this.price = price; this.quantity
= quantity;
}
// Getters & Setters public int getId()
{ return id; } public void setId(int id) {
this.id = id; }

public String getName() { return name; }
public void setName(String name) { this.name = name; }

public double getPrice() { return price; }
public void setPrice(double price) { this.price = price; }

public int getQuantity() { return quantity; } public void
setQuantity(int quantity) { this.quantity = quantity; }
}
```

```
ProductController.java package myproject; import
org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Controller; import
org.springframework.ui.Model; import
org.springframework.web.bind.annotation.*;
```

```
@Controller public class
ProductController {
    @Autowired
    ProductRepository repo;

    @GetMapping("/")

    public String home(Model model) {
model.addAttribute("products", repo.findAll());
return "index"; // must match file name
    }
}
```

```

    @PostMapping("/add")
    public String add(Product p) {
        repo.save(p);    return
        "redirect:/";
    }

    @GetMapping("/delete/{id}")    public
    String delete(@PathVariable int id) {
        repo.deleteById(id);    return "redirect:/";
    }

    @GetMapping("/edit/{id}")    public String edit(@PathVariable int id,
    Model model) {    model.addAttribute("product",
    repo.findById(id).orElse(new Product()));    return "edit";
    }

    @PostMapping("/update")
    public String update(Product p) {
        repo.save(p);    return
        "redirect:/";
    }
}

```

● ProductRepository.java

```

package myproject; import
org.springframework.data.jpa.repository.JpaRepository; public interface
ProductRepository extends JpaRepository<Product, Integer> { }

```

● SecurityConfig.java

```

package myproject; import
org.springframework.context.annotation.Bean; import
org.springframework.context.annotation.Configuration; import
org.springframework.security.config.annotation.web.builders.
HttpSecurity; import
org.springframework.security.web.SecurityFilterChain;
@Configuration public class
SecurityConfig {

```

```

@Bean    public SecurityFilterChain filterChain(HttpSecurity http) throws
Exception {
    http
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(auth -> auth
            .anyRequest().permitAll()
        );
    return http.build();
}
}

```

○ User.java

```

package myproject; import
jakarta.persistence.*;
@Entity public
class User {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int id;    private
String username;    private
String password;
    // getters & setters
}

```

○ UserRepository.java

```

package myproject;
import org.springframework.data.jpa.repository.JpaRepository; import
myproject.User; public interface UserRepository extends
JpaRepository<User, Integer> {
    User findByUsername(String username);
}

```

```
}
```

○ edit.html

```
<!DOCTYPE html>

<html xmlns:th="http://www.thymeleaf.org">

<head>

<link rel="stylesheet" href="/css/style.css">

</head>

<body>

<div class="login-box">

<h2>Edit Product</h2>

<form action="/update" method="post">

<input type="hidden" name="id" th:value="${product.id}">

<input name="name" th:value="${product.name}">

<input name="price" th:value="${product.price}">

<input name="quantity" th:value="${product.quantity}">

<button>Update</button>

</form>

</div>

</body>

</html>
```

○ index.html

```
<!DOCTYPE html>

<html xmlns:th="http://www.thymeleaf.org">

<head>

<title>Inventory Dashboard</title>

<link rel="stylesheet" th:href="@{/css/style.css}">

<!-- Chart.js -->

<script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
```

```
</head>

<body>

<div class="container">

  <h1>Inventory Dashboard</h1>

  <!-- ADD PRODUCT FORM -->

  <form action="/add" method="post" class="form">

    <input type="text" name="name" placeholder="Product Name" required>

    <input type="number" step="0.01" name="price" placeholder="Price" required>

    <input type="number" name="quantity" placeholder="Quantity" required>

    <button type="submit">Add Product</button>

  </form>

  <!-- PRODUCT TABLE -->

  <table>

    <thead>

      <tr>

        <th>Name</th>

        <th>Price</th>

        <th>Quantity</th>

        <th>Status</th>

        <th>Action</th>

      </tr>

    </thead>

    <tbody>

      <tr th:each="p : ${products}">

        <td th:text="${p.name}"></td>

        <td th:text="${p.price}"></td>

        <td th:text="${p.quantity}"></td>

        <!-- STATUS LOGIC -->

        <td>

          <span th:if="${p.quantity == 0}" class="out">Out of Stock</span>

        </td>

      </tr>

    </tbody>

  </table>

</div>

</body>

</html>
```

```
        <span th:if="{p.quantity > 0 and p.quantity <= 5}" class="low">Low
Stock</span>
```

```
        <span th:if="{p.quantity > 5}" class="in">In Stock</span>
```

```
    </td>
```

```
    <td>
```

```
        <a th:href="@{'/edit/' + ${p.id}}">Edit</a>
```

```
        <a th:href="@{'/delete/' + ${p.id}}">Delete</a>
```

```
    </td>
```

```
</tr>
```

```
</tbody>
```

```
</table>
```

```
<!-- CHART SECTION -->
```

```
<h2>Stock Overview</h2>
```

```
<div class="chart-container">
```

```
    <canvas id="stockChart"></canvas>
```

```
</div>
```

```
</div>
```

```
<!-- CHART SCRIPT -->
```

```
<script th:inline="javascript"> let
```

```
products = /*[[${products}]]*/ []; let
```

```
inStock = 0; let lowStock = 0; let
```

```
outStock = 0; products.forEach(p => {
```

```
if (p.quantity == 0) outStock++; else
```

```
if (p.quantity <= 5) lowStock++; else
```

```
inStock++;
```

```
});
```

```
new Chart(document.getElementById("stockChart"), {
```

```
    type: 'bar',    data: {    labels: ['In Stock',
```

```
'Low Stock', 'Out of Stock'],    datasets: [{
```

```
label: 'Products',    data: [inStock, lowStock,
```

```

    outStock],          backgroundColor: ['green',
    'orange', 'red']
        }],      },      options: {
responsive: true,      plugins: {
legend: { display: false }
        }
    }
});
</script>
</body>
</html>

```

○ login.html

```

<!DOCTYPE html>

<html xmlns:th="http://www.thymeleaf.org">

<head>

    <title>Login</title>

    <link rel="stylesheet" th:href="@{/css/style.css}">

</head>

<body>

<div class="login-container">

    <div class="login-box">

        <h2>Login</h2>

        <form action="/login" method="post">

            <input type="text" name="username" placeholder="Username" required>

            <input type="password" name="password" placeholder="Password" required>

            <button type="submit">Login</button>

        </form>

    </div>

</div>

```


</body>

</html>

➤ **applications.properties**

spring.datasource.url=jdbc:mysql://localhost:3306/inventory

spring.datasource.username=root

spring.datasource.password=nithish@123

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true

server.port=8099